

Proyecto Final

Desarrollar un sistema de gestión de inventario para una gran superficie que organiza sus productos en categorías y subcategorías de profundidad variable. Deberás diseñar la lógica mediante **diagramas de flujo** y programar una aplicación que permita administrar productos, procesar datos y buscar información mediante algoritmos recursivos.

Estructura de Datos Base (Anidamiento y Listas)

El sistema operará con una estructura de datos en memoria utilizando diccionarios y listas anidadas:

Ejemplo:

```
inventario = [  
  {  
    "categoria": "Electrónica",  
    "subcategorias": [  
      {  
        "categoria": "ordenador",  
        "productos": [  
          {"id": 1, "nombre": "ordenador X", "precio": 1200, "stock": 5},  
          {"id": 2, "nombre": "ordenador Y", "precio": 900, "stock": 0}  
        ]  
      }  
    ]  
  },  
  {  
    "categoria": "Hogar",  
    "productos": [  
      {"id": 4, "nombre": "Cafetera", "precio": 80, "stock": 12}  
    ]  
  }  
]
```

Requisitos del Proyecto

1. Fase de Diseño: Diagrama de Flujo

Debes presentar un diagrama de flujo que detalle:

- **La función de búsqueda recursiva:** Cómo el algoritmo decide si profundizar en una subcategoría o si ha encontrado el producto (caso base).
- **El flujo del menú principal:** El ciclo de vida de la aplicación, desde la entrada del usuario hasta la gestión de errores y salida.

2. Gestión y Lógica

- **Añadir/Actualizar:** Funciones modulares para insertar productos.

- **Validación:** Uso de condicionales para asegurar que los precios y existencias sean lógicos (no negativos).

3. Algoritmos Avanzados

Debes implementar:

- **Búsqueda recursiva:** `buscar_producto(inventario, id_producto)` que devuelva el objeto si existe.
- **Valor total:** Una función recursiva que calcule el valor monetario de todo el inventario sumando todas las ramas.

4. Procesamiento de Datos (map y filter)

- **Informe Crítico:** Uso de `filter` para obtener productos con `stock < 5`.
- **Descuentos:** Uso de `map` para aplicar descuentos porcentuales a una lista de productos sin mutar los datos originales.

5. Manejo de Excepciones

- Uso de `try...except` para capturar errores de entrada de datos (`ValueError`).
- Creación y lanzamiento (`raise`) de una excepción personalizada `ProductoNoEncontradoError`.

Entregables

1. **Documento de Diseño:** Archivo (PDF o imagen) con los diagramas de flujo solicitados.
2. **Código Fuente:** Archivo `main.py`.

Criterios de Evaluación

Criterio	Descripción	Puntuación
Diseño Lógico	Claridad y exactitud del diagrama de flujo (especialmente en la recursión).	15%
Lógica y Funciones	Uso de condicionales, modularidad y cumplimiento del principio DRY.	15%
Estructuras Anidadas	Manipulación correcta de la jerarquía de listas y diccionarios.	15%
Paradigma Funcional	Aplicación correcta de <code>map</code> y <code>filter</code> con funciones <code>lambda</code> o regulares.	15%
Recursividad	Implementación efectiva de casos base y llamadas recursivas.	20%

Criterio	Descripción	Puntuación
Manejo de Errores	Gestión de excepciones estándar y personalizadas para un programa robusto.	20%

Nota: El proyecto no será evaluado si el código no se corresponde con la lógica planteada en el diagrama de flujo entregado.